DESIGN NOTE · FOR REVIEW

Golden Thread

A layered knowledge architecture for Claude Code. Keeping every one of your projects current with what you have figured out — without hand-carrying each refinement to them one at a time.

Stacy Haven with input from **Jonathan Tucci**

STATUS Draft — seeking critique DATE 9 August 2026 DESIGNED FOR One developer, many projects

THE PROBLEM

You become the bottleneck

Working agreements with an AI assistant do not arrive fully formed — they accumulate. Over weeks on a project the shared understanding sharpens: always back a file up before overwriting it, never restart that daemon without asking, this is what "done" means here. Every one of those refinements is real progress, and every one of them is learned in exactly one place.

Then you open a second project. Or come back to one you set down in March. The refinements do not come with you, so you explain them again — and not once. Each addition has to be carried to every project it applies to. Each change of mind has to be chased down everywhere you already taught the old version. The effort scales with projects multiplied by revisions, and neither number ever goes down.

Subtraction is the worse half. Failing to add a rule leaves a project merely less capable. Removing one you have since decided against means finding every session you already taught it to — and the ones you miss carry on confidently doing the thing you stopped wanting, which is far harder to notice than an omission and more expensive by the time you do.

The uncomfortable part is that **you are the transport layer**. Knowledge moves between your own projects at the speed of you remembering to move it, and the true cost of every new insight includes the tax of redistributing it by hand. That is the bottleneck this design removes: not the storing of what you have learned, but the standing obligation to keep every parallel session in agreement with what you currently believe.

This is a design for **one developer with many projects**. The hard problem is not coordinating people — it is that one person's understanding keeps improving and has no way to reach the work already underway elsewhere. It works just as well for several developers sharing context — across one project or many — because every boundary in the design is drawn between machines and projects, never between people. That case is fully supported; it simply isn't the one that hurts most.

The obvious fix — put everything into every project — fails for its own reason: instructions loaded at the start of a session cost context in *every* session, and most of what one project knows is noise to another. Broadcasting does not scale, and neither does explaining yourself repeatedly.

What follows is an architecture for sharing the parts that generalise, keeping the parts that do not, and retrieving the rest only when asked.

It is called **Golden Thread** after the idiom for the continuity that runs through an entire body of work — the strand you can follow from one end of it to the other. That is the thing the design is trying to produce: not a store of documents, but a line of reasoning that survives being carried from one project to the next.

STARTING POINT

What already worked, and where it stopped

One machine already mirrored its Claude memory directory into an Obsidian vault: a cron job copied memory files into folders, regenerated an index, committed, and pushed to a private GitHub repository. Roughly 160 notes, each holding a single fact,

cross-linked as a graph.

It worked well because it was simple, and it was simple because it made three assumptions: **one source machine, one direction, one writer.** Memory flowed out to the mirror and nothing ever flowed back. Those assumptions are exactly the ones that break when a second machine wants in — and the failure is quiet. Two machines regenerating the same index from different views of the truth would erase each other's entries on every run, forever, and no retry logic helps because it is not a race. It is two writers with different ideas of the whole.

> THE GOVERNING INSIGHT
>
> The shared repository is a **mirror**, not a source. Every design decision below follows from that: editing the mirror achieves nothing, because the next sync overwrites it from the source. Changes have to happen upstream or they do not happen at all.

THE MODEL

Four kinds of thing, and only one of them is always paid for

The first mistake was treating everything in the system as "notes." Separating them by *how they reach a session* settles the argument about context cost, because it turns out only one kind has to be there before anyone asks.

KIND	REACHES A SESSION BY	STANDING COST	EXAMPLE
Constraint	**always loaded**	**paid every session**	**never restart production without asking**
Procedure	**a skill that fires on intent**	**a one-line description**	**how to run a backtest reset**
Knowledge	**the Librarian, on request**	**none until asked**	**how the relay pipeline works**
Source	**dug into behind knowledge**	**none, ever**	**the actual API response, captured verbatim**

Constraints are what must be true before work starts, so they load. Procedures are recipes, so they become skills whose descriptions are always visible but whose instructions load only when the description matches what is happening. Knowledge is fetched. Sources are never loaded at all — they are what knowledge cites when a number has to be exact.

Five layers, filed by who needs it

A single "shared" bucket forces a false choice: either a rule is universal or it is local. Most real rules are neither. The rule that a trading model may never use information it could not have known at the time is absolute across every trading system — and meaningless on a box that manages network hardware. A **session** is one Claude project or working directory; an **area** is one subsystem inside it.

`MACHINE` **One box, whatever you are working on** `_Machines/<host>/`

Facts that follow the hardware, not the work: where a browser is installed, which fonts exist, what this box cannot resolve. They had nowhere to live before and squatted inside whichever project happened to discover them.

`CORE` **Everywhere, always** `_Core/`

Fourteen constraints. Loaded into every project on every machine, which is exactly why it is kept to one line each — this is the only layer everyone pays for continuously.

`SESSION` **One session, all of its areas** `<Session>/_Rules/`

The no-look-ahead rule lives here: absolute within its world, absent outside it.

`AREA` **One subsystem** `<Session>/<Area>/`

Version-bump conventions, a specific file's gotchas, and the incidents that produced them.

`REPO` **The codebase and everyone who clones it** `the repo itself — outside this system`

The exit. A fact that would make sense to someone who has never heard of this vault does not belong in it. It graduates into the codebase's own instructions by pull request, and the note here retires with a pointer.

> WHY THE EXIT MATTERS
>
> Without it, every mechanism here funnels knowledge inward and nothing ever leaves. A system with no exit hoards team knowledge in a place exactly one person reads. Six of the notes reviewed turned out to belong in codebases rather than here.

Applied to an existing vault, two of these already existed. The folder labelled "applies across every project" had always meant every *area* within one memory store — it was never claiming to be universal. What was missing was a layer above it, so genuinely global rules had nowhere to go, and a layer below it for the machine, so hardware facts squatted in project folders.

Rules do not sort cleanly — they need splitting

Reviewing 28 accumulated rule-notes, the dominant pattern was not "universal" or "local" but **a universal rule wrapped around a local incident**:

THE NOTE	UNIVERSAL RULE INSIDE IT	LOCAL EVIDENCE
Silent functions	**If a bug trail reaches code with no logging, adding logging *is* part of the fix — "no root cause, but I made it observable" is a legitimate outcome**	**A specific daemon, a specific 90-minute stall**
Validate against the source	**Never validate against a hand-copied constant; parse the live artifact — copies drift silently**	**A preset dictionary missing seven parameters**
Daemons via service manager	**Never start a persistent process with a bare background shell command**	**Three named hosts, three audited exceptions**

No depth of hierarchy fixes a note that carries two altitudes. The rule gets extracted upward; the evidence stays where it happened and is linked from above. Of 28 notes, fourteen become Core constraints — four of those by splitting a universal rule away from the machine or area fact welded to it — eight are session-wide, and six turn out to belong in a codebase rather than here at all.

Two tests make that split reproducible instead of a matter of taste. **One home per sentence:** a line that could sit in two layers is a defect, and the fix is almost always to separate the general lesson from the incident that taught it. **The teammate test:** if a line would make sense to someone who has never heard of this system, it is a repo fact and belongs in the repo. Applying the first caught all four splits above without anyone exercising judgement.

PROVENANCE

Notes answer concepts; sources answer specifics

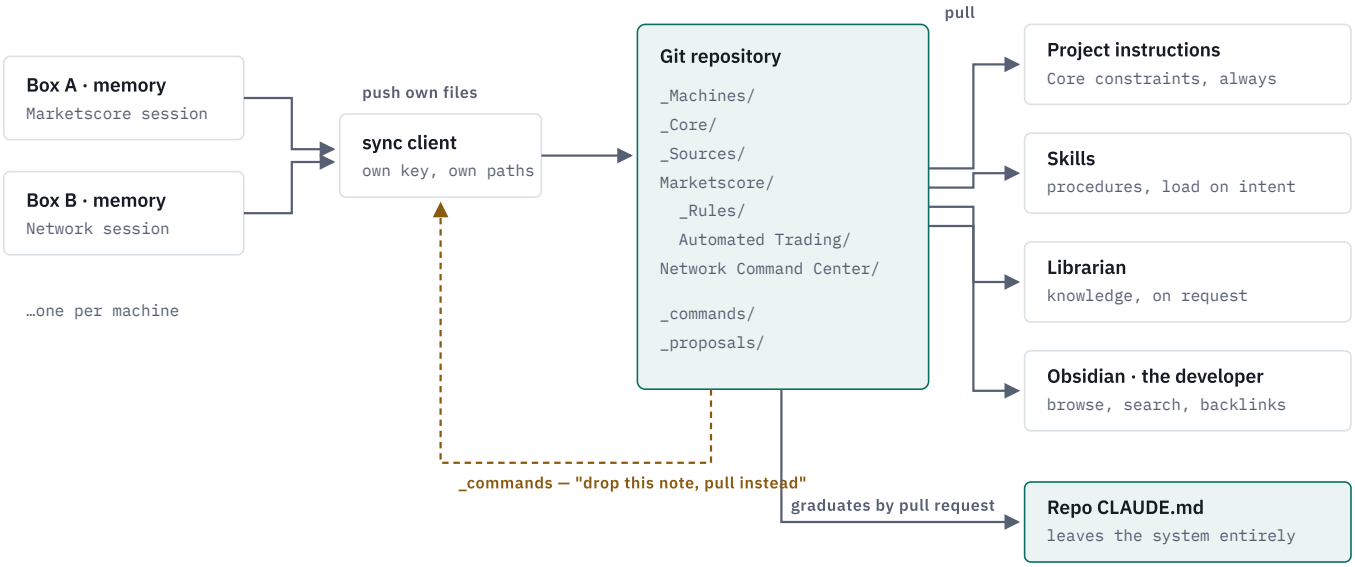
A note claiming that a whitelist lives in a particular file, or that a limit is a particular number, is a claim with a shelf life. When it goes stale there is nothing behind it to re-derive from — the original material was read once, summarised, and thrown away.

So raw material is kept, in `_Sources/`, exactly as captured and **never edited afterwards**: config files, API responses, specs, transcripts. Notes cite the sources they were built from. When a question needs an exact path, a rate limit or a configuration value, the lookup goes through the note into the source behind it, always.

Immutability is what makes it worth having. An edited source is just another note with a confusing name; an untouched one is evidence. When upstream changes, a new source supersedes the old rather than overwriting it, and the notes citing it get updated to match.

TOPOLOGY

How it moves



Each box writes only its own paths with its own credential; nothing reads its peers. Four readers consume the repository, and they differ in what they cost: always-loaded constraints, skills that load on intent, the Librarian on request, and the developer's own Obsidian window. The path at the bottom is the exit — a fact that belongs to a codebase leaves the system rather than accumulating in it. The dashed return is the only channel that reaches back into a machine, and it carries data, never code.

Every box is its own writer

The tempting alternative was a single publisher: one machine collects everything over SSH and pushes on everyone's behalf. It is simpler and removes write contention entirely. It was rejected on security grounds, and the reason is worth stating precisely — it is not about credentials.

> WHY NOT A CENTRAL PUBLISHER
>
> A central publisher needs standing SSH read access into every other machine's Claude memory directory: a permanent cross-network capability over everything every AI assistant has ever learned on those boxes. Per-box writers delete that entirely. Each machine talks only to the repository and never to its peers, so compromising one yields nothing about the others.

The cost is write contention, handled conventionally: each box owns a disjoint set of paths, so concurrent commits touch disjoint files and rebase cleanly; pushes fetch, rebase and retry. There is no shared index file — each session writes its own, and only the repository owner writes the root.

One honest limitation: **deploy keys are scoped per repository, not per path.** A key with write access can write anywhere in the repo. Path ownership is a convention the clients honour, not a boundary the host enforces. Enforcing it would mean a repository per session, or branch protection with review — a real trade against the convenience of one graph.

THE PROTOCOL

Promotion, and the problem it creates

Promoting a rule from Session to Core means moving a note between folders. Do that in the shared repository and it silently reverts within the hour: the source machine's sync still finds the note listed in its own index, still finds the file in its memory directory, and copies it faithfully back. Worse, the same filename now exists in two folders — and in a wiki-linked vault where links resolve by filename, every reference to it becomes ambiguous.

So promotion has to happen at the source: the note leaves the originating machine's memory entirely, and that machine receives the rule back by *pulling* Core, like everyone else. Which requires telling it to stop pushing something it currently believes it owns.

A commands channel that carries data, not code

The natural implementation is a script in the repo that each box runs on sync. That is also a remote code execution channel: anything able to write the repository runs arbitrary code on every machine that pulls it. The portability argument against shell scripts is real; this one matters more.

Instead, a declarative manifest with a small closed vocabulary:

```
_commands/Marketscore.json
{
  "revision": 7,
  "issued": "2026-08-09T15:12:33Z",
  "actions": [
    { "action": "promote",
      "note": "feedback_timestamp_every_message.md",
      "from": "Marketscore/Cross-Cutting",
      "to":   "_Core Mandate",
      "drop_from_session_memory": true }
  ]
}
```

- **Revision numbers give idempotency.** Each box records the last revision it applied locally, so re-running costs nothing and a missed sync self-heals on the next one.
- **The vocabulary is closed** — `promote`, `retire`, `relocate`, `refresh`. Anything unrecognised is ignored and logged, so an older client cannot be steered into behaviour it does not understand.
- **Reading costs nothing in ownership.** The owner writes the manifest; the box only reads it, so no partition is violated.
- **Acknowledgements go in the box's own folder**, which keeps partitioning intact and yields a view of which machines are current.

The reverse channel is `_proposals/<session>/`: a session that believes it has found something universal drops a note there rather than writing to Core. Each session owns its own subfolder, so proposals never collide, and promotion stays a deliberate decision rather than a race to claim the shared namespace.

Nothing is edited; things are superseded

A rule that has gone stale is one problem. A rule that was simply *wrong* is a different one, and for a long time this design had no answer to it. Correcting a note in place looks like the obvious fix and destroys the only evidence of why anyone believed it — which matters most when the question resurfaces a year later and the reasoning has to be re-examined rather than rediscovered.

So notes are never edited into correctness. A new note supersedes the old, the old is marked `superseded_by` and stays readable, and the record shows both what was believed and what changed it. Retirement stops meaning deletion. Two command actions carry it: `supersede`, and `graduate` for a fact leaving through the repo exit.

RETRIEVAL

Four readers, one substrate

The same repository is consumed four different ways, and the format has to serve all of them at once. Three of the readers are the AI assistant; the fourth is the person.

1 · Standing instructions — broadcast

Core, and only Core, is written into every project's instruction file and loaded automatically at the start of every session. It is the one level worth what it costs everywhere, which is exactly why it is kept to fourteen one-line constraints.

2 · Skills — loaded on intent

Procedures are not constraints and should not be paid for continuously. Each becomes a skill: a description that is always visible, and instructions that load only when the description matches what is actually happening. The standing cost is a line or two; the body arrives when it is relevant.

Two rules keep the set coherent. Skills compose through files, never by calling each other, so removing any one leaves the rest working. And no two may plausibly fire on the same intent, which is checked whenever one is added — because writing the trigger description *is* the engineering. A description that fires on nothing is worse than no skill at all, since it looks like coverage.

3 · Lookups — on demand

Everything else is retrieved only when needed: the project asks a question, a short-lived AI assistant scoped to the thread searches it, answers, and exits. Zero context cost until the moment of need. There is no persistent overseer holding it all in mind — call it the **Librarian**, a stateless function over a directory of markdown, invoked per question. That sounds like a limitation and is mostly a feature: it is cheap, parallel, and cannot drift from what is actually on disk.

> THE USEFUL SIDE EFFECT
>
> Log the questions. When several projects independently ask something that an area-level note already answers, that is the promotion queue writing itself — evidence of generality, rather than a judgement call about it. Usage decides what is universal.

4 · The developer's own window — Obsidian

The third reader is the one the whole thing is for. Every file is plain markdown with wiki-style links, so the repository opens directly as an Obsidian vault on the developer's own machine — no export step, no custom viewer. What that buys is the view no single project can give you: full-text search across *every* project at once, backlinks showing which other work referenced a finding, and a graph in which a note written during one project visibly connects to another. Cross-project connections stop being a thing you have to remember and become a thing you can see.

Keeping it current needs no OS-level scheduler: the `obsidian-git` community plugin pulls, commits and pushes on an interval from inside the app, which behaves identically on macOS and Linux and asks nothing of the machine it runs on.

> A TRAP WORTH DESIGNING AROUND
>
> Reading in Obsidian invites editing in Obsidian — and edits to a mirror are silently discarded on the next sync, exactly like the promotion problem above. The local copy should be treated as read-only by the human, with one exception: an `_inbox/` folder that no sync ever overwrites, where a thought can be dropped safely and filed properly later.

That third reader is also why the substrate is plain markdown rather than a database. A database would serve the two machine readers better in every respect — indexing, querying, integrity — and would lock the person out entirely. The format is chosen for the reader who cannot be reprogrammed.

CLEANUP

Keeping it honest

The dangerous failure of a knowledge system is not the thing you never captured. It is the thing you captured, kept, and still serve after reality moved on — because a session reads it and follows it with complete confidence. Adding is solved several ways here. Removing had to be designed on purpose.

Type the note, so only half of it ever needs reviewing

Every note is typed when written: **principle** or **fact**. A principle — never restart production without asking — does not decay, and reviewing it forever is wasted effort. A fact names something: a path, a host, a config key, a line number. Facts rot. Only they get checked, which roughly halves the standing review burden before anything else is done.

Deterministic detection, model proposal, human approval

Finding problems is a script's job, not a judgement call. Broken links, orphan notes, index mismatches, duplicate basenames, and referents that no longer exist on disk are all mechanically checkable. Code finds them, the assistant proposes a fix, a person approves it. That split also closes two of the gaps in the scenario map — renames and filename collisions — without any inference at all.

Four signals, and one that is explicitly rejected

A fact becomes a review candidate when its referent no longer exists, its declared `expires_when` condition is met, it contradicts another note retrieved alongside it, or you flag it in the moment you catch a session citing something out of date. That last one starts with a person and is the strongest evidence available, because it arrives exactly when the truth is known.

> NOT A SIGNAL
>
> "Nobody has asked about this note" seems like the obvious measure and is actively harmful. A rule nobody asks about is either obsolete or so thoroughly followed that no question ever arises, and those are indistinguishable in usage data. Retiring on silence removes the best rules first.

The queue, and why it is not another dashboard

Candidates accumulate with their evidence attached and arrive as a single daily email — items inline, so most can be judged without opening anything. Contradictions are the exception and send immediately, because two rules actively disagreeing is worth knowing today. A per-item notifier would become noise within a week, and a queue you have learned to ignore is worse than no queue, since it looks like coverage.

Promotion fires on events rather than telemetry — a fact that survived several sessions unchanged, a finding that held up at ship time, the same issue recurring. That matters for sequencing as much as for correctness: it means the review loop works

before any query logging exists, so the most valuable part of the system does not have to wait on the Librarian.

WHERE I WANT ARGUMENT

Open questions

Three of the original six are now answered and have moved into the design above: promotion triggers on events, retirement works by supersession, and the context budget is handled by loading only constraints. What remains is genuinely unresolved, and is the reason for circulating this.

Q1 **Is convention-based path ownership good enough?**

Every box can technically write every path; deploy keys are scoped per repository, not per directory. One repository per session would enforce the boundary but fragments the graph, and cross-session links are the entire point. Is there a middle option — signed commits, a verifying hook, required review — that keeps one graph without trusting every client?

Q2 **Which wins when two layers disagree?**

The working assumption is that the more specific layer overrides the more general one, which matches how people reason but makes a Core rule a default rather than a mandate. If Core is genuinely inviolable, a conflict should be an error raised at sync time instead of a precedence rule applied silently.

Q3 **A note that is unlisted is invisible, and nothing says so.**

Publication is driven by an index. A note written to disk but never added to it is simply never published, and no error is raised — it just quietly does not exist. Detecting that is easy; deciding whether the index or the directory is authoritative is not.

Q4

Mirror or source?

This design mirrors per-machine memory into the repository rather than having sessions write to it directly. The mirror is what makes multiple machines possible — but it is also the direct cause of deletions never propagating, renames leaving duplicates, and the whole promotion protocol. A single-machine system would not need any of it. Is multi-machine worth that much machinery?

Q5

Who does the gardening when it is not one person?

Every quality mechanism here ends at one human approving a promotion. That is fine for one developer and is exactly what fails to scale. A team version that shares the data without also sharing the approval will rot politely and look healthy while doing it.

Q6

Could this be useful to anyone else?

Nothing here is domain-specific: markdown, git, a small manifest format, and a client that is deliberately portable. Is a general tool worth extracting, or is the interesting part the discipline rather than the code?

SEQUENCING

Build order

Each step is safe on its own and useful before the next one lands.

01 **Done.** Portable client — stdlib Python, POSIX and Windows paths, no shell dependency. Pulls Core into the managed block, reports pending commands, records what it applied.

02 Lint: broken links, orphans, dead referents, index mismatches, duplicate basenames. Read-only, and the fastest way to find out what is already broken.

03 The first Core notes — the protocol itself, the two filing tests, and source control as a precondition.

04 Per-box publisher: partitioned paths, fetch, rebase, push retry, per-session index.

05 Restructure the existing notes into the five layers, with the sources tier alongside.

06 Promote the Core notes *at the source*, not in the mirror.

07 One deploy key per box; retire the central scheduled job.

08 The review queue and its daily digest — event triggers, no telemetry required.

09 Librarian retrieval, index-first and link-following, with question logging.

10 Obsidian on the developer's machine, plus the `_inbox/` the sync never touches.

> **PREREQUISITE, NOT A STEP**
>
> A project must be under source control before this attaches to it. The client writes into the project — a managed block, skill files, state — and without history there is no diff and no undo, which contradicts the very rules it is distributing. That makes source-control migration a gate on adoption rather than an adjacent improvement, and for an existing codebase it starts with a credential scan: committed secrets cannot be edited out, only rewritten everywhere and rotated.

> **RISKIEST STEP**
>
> Restructuring moves every note in a live mirror while a scheduled job is writing to it. The scheduler has to be paused and the folder mapping updated in the same window, or the next run re-creates the old layout alongside the new one and duplicates the collection.

The layered filing, the immutable sources tier, supersession in place of editing, skills as the loading mechanism, and the insistence that knowledge must be able to leave the system all come from Jonathan Tucci's Mindforge — a system of the same shape that has been in daily use for months. Two designs reached

markdown, git, human-approved promotion and cleanup-as-the-neglected-half independently, which is the strongest validation either has.

A companion document maps every scenario the system can be in, drawn as flows, including the ones that currently have no path.

Draft for circulation. Hostnames, addresses and domain-specific rule content have been generalised; the structure is exactly as built.